PROJETO CYLAB · PUC-CAMPINAS

Cylab: um conjunto de dados ouro de pares vulnerável–corrigido verificados por análise estática para um assistente local de segurança de código

Relatório técnico — 1ª entrega (dataset ouro FINAL v0.4-gold, 576 pares)

Arthur Alcausa e equipe do Projeto Cylab
Pontifícia Universidade Católica de Campinas (PUC-Campinas)

Campinas, 28 de outubro de 2026

Repositório público: https://github.com/ArthurAlcausa/cylab-dataset

RESUMO

Ferramentas de análise estática de segurança (SAST) apontam falhas, mas raramente explicam o problema em português ou propõem uma correção adaptada ao código do desenvolvedor. Assistentes baseados em modelos de linguagem de grande porte (LLMs) hospedados em nuvem suprem parte dessa lacuna, porém exigem o envio do código-fonte a terceiros e podem relatar vulnerabilidades inexistentes. Este relatório descreve a primeira entrega do Projeto Cylab, que adota uma arquitetura híbrida: um motor determinístico (Opengrep) detecta a falha, um LLM executado localmente explica e propõe o *patch*, e o mesmo motor verifica o *patch*. O resultado desta etapa é um conjunto de dados ouro (FINAL v0.4-gold) com 576 pares vulnerável–corrigido em 10 linguagens, escritos do zero sob licença CC-BY-4.0. Cada par satisfaz um critério de aceite automático (mais de zero achados da regra no código vulnerável e zero no código corrigido); a verificação registrou 576/576 aprovações. O conjunto resultou de um funil de qualidade que partiu de 2.191 pares (v0.2), removeu quase-duplicatas e produziu o corte anterior FINAL v0.3-gold (476 pares); a v0.4-gold acrescenta a ele um *buffer* de realismo de 100 pares com funções maiores (mediana de 35 linhas no trecho vulnerável), sem alterar nenhum registro da v0.3. O conjunto e os *scripts* estão publicados em repositório público (CYLAB, 2026l). Ainda não há modelo treinado; portanto, nenhuma métrica de modelo é reportada. Discutem-se as limitações (pares sintéticos ainda predominantemente curtos, verificação por regra não equivalente a correção semântica, ausência de análise de fluxo em C/C++) e o plano de ajuste fino e avaliação.

Palavras-chave: segurança de software; análise estática; SAST; modelos de linguagem; conjunto de dados; CWE.

1 INTRODUÇÃO

1.1 Contexto e problema

A detecção automatizada de vulnerabilidades em código-fonte é tradicionalmente feita por ferramentas de análise estática (SAST), que aplicam regras sintáticas ou de fluxo de dados e associam cada achado a uma categoria da *Common Weakness Enumeration* (CWE) mantida pela MITRE (MITRE CORPORATION, 2026a). Ferramentas como o Semgrep e seu *fork* Opengrep oferecem regras declarativas em YAML que casam padrões estruturais de código (SEMGREP, 2026; OPENGREP, 2026a). Esses achados, contudo, são acompanhados de mensagens curtas, geralmente em inglês, e não trazem uma correção para o trecho analisado.

Este trabalho trata de três problemas, conforme o roteiro do projeto (CYLAB, 2026a):

1. **Explicação e correção.** O SAST indica onde está o problema, mas não explica em português do Brasil nem propõe uma reescrita do trecho.
2. **Privacidade.** Assistentes de código baseados em LLMs hospedados em nuvem exigem o envio do código-fonte, às vezes proprietário, a terceiros, o que muitas organizações não podem fazer.
3. **Confiabilidade.** Um LLM usado isoladamente como detector pode relatar vulnerabilidades inexistentes e deixar de relatar outras, sem um critério auditável.

A questão norteadora é: *é possível oferecer explicação e correção em português, sem que o código saia da máquina do usuário e sem delegar ao LLM a decisão sobre o que é vulnerável?* (CYLAB, 2026a).

1.2 Objetivo desta entrega

Esta primeira entrega não apresenta um produto acabado nem um modelo treinado. Seu objetivo é (i) definir a arquitetura híbrida e (ii) construir e validar o conjunto de dados que servirá para treinar e avaliar o componente de linguagem. O escopo de linguagens compreende Python, JavaScript/TypeScript, Java, PHP, C#, C, C++, SQL e Shell/Bash (CYLAB, 2026a), o que corresponde a 10 valores distintos no campo `lang` do conjunto de dados (CYLAB, 2026m). O conjunto de dados, as regras, os *scripts* de verificação e este relatório estão disponíveis no repositório público https://github.com/ArthurAlcausa/cylab-dataset (CYLAB, 2026l).

1.3 Organização do texto

A Seção 2 descreve a fundamentação e o método: arquitetura híbrida, critério de aceite, regras endurecidas, detecção de quase-duplicatas e a onda de realismo da v0.4. A Seção 3 apresenta os materiais, isto é, o esquema dos pares e o processo de curadoria. A Seção 4 traz os resultados quantitativos. A Seção 5 discute as limitações. A Seção 6 descreve os trabalhos futuros e a Seção 7 conclui. As referências seguem a ABNT NBR 6023 e incluem os artefatos do repositório como documentos técnicos.

2 FUNDAMENTAÇÃO E MÉTODO

2.1 Delineamento da pesquisa

Trata-se de pesquisa aplicada, de caráter tecnológico, conduzida em ciclos curtos de construção e verificação de artefatos. Cada ciclo produz uma versão do conjunto de dados, submete-a a critérios automáticos (validador de esquema, *gate* Opengrep e detecção de quase-duplicatas) e registra os resultados em arquivos JSON versionados no repositório

(CYLAB, 2026l). Uma versão só é promovida a "ouro" quando todos os critérios passam e o resultado é reprodutível a partir dos *scripts* publicados. Os números deste relatório são, portanto, medidas de consistência do conjunto de dados, e não de desempenho de modelo.

2.2 Arquitetura híbrida

A arquitetura proposta separa detecção, explicação e verificação (CYLAB, 2026a). A Figura 1 resume o fluxo: o código-fonte do desenvolvedor é varrido pelo Opengrep com as regras Cylab, cada uma associada a uma CWE; o achado resultante é entregue ao LLM local, servido pelo Ollama, que produz uma explicação em português e um *patch*; o *patch* é varrido novamente pelo Opengrep, e só é aceito se passar no *gate*. Todo o fluxo ocorre na máquina do usuário.

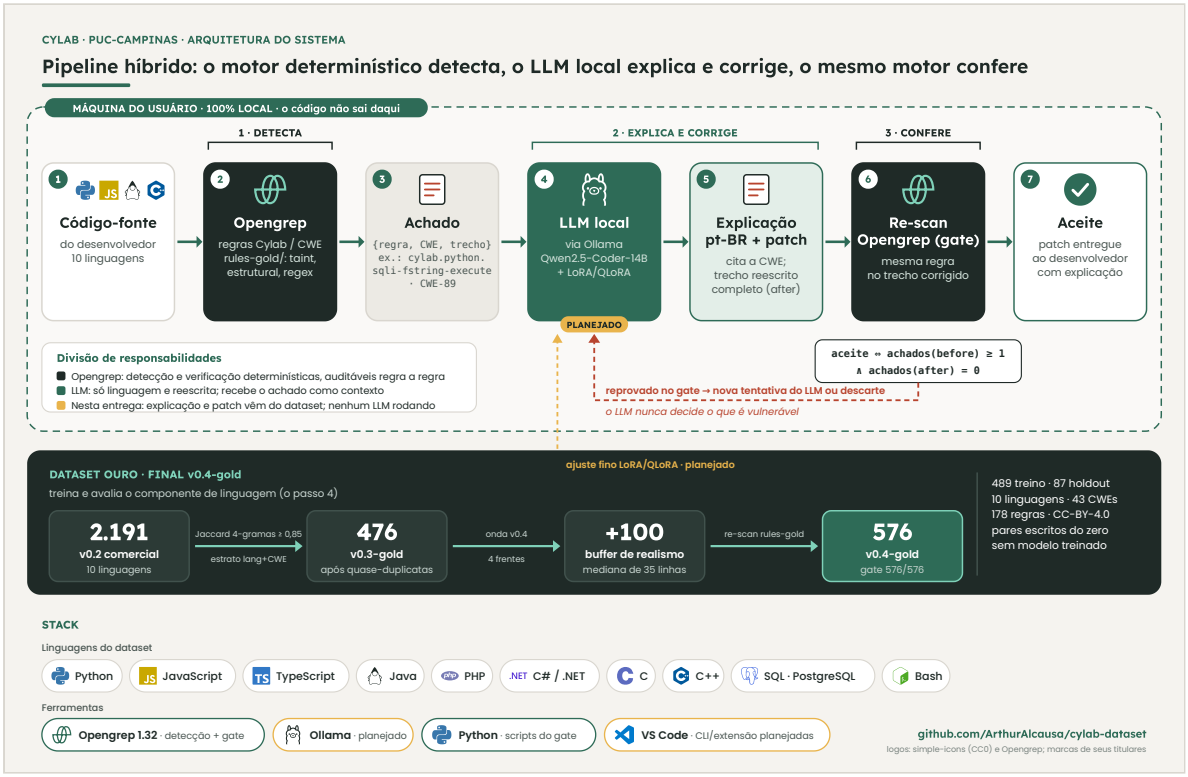


Figura 1 — Arquitetura híbrida do Cylab: detecção determinística (Opengrep), explicação e correção por LLM local (Ollama, planejado) e re-varredura como critério de aceite; abaixo, o funil do conjunto de dados ouro e a *stack* tecnológica. Fonte: elaborada pelos autores (seminar/assets/cylab-arquitetura.svg).

Em forma textual, o mesmo fluxo é:

```
código-fonte → Opengrep (regras Cylab / CWE) → achado {regra, CWE, trecho}
             → LLM local (Ollama) → explicação pt-BR + patch
             → re-scan do patch com Opengrep (gate) → aceite
               (reprovado no gate: nova tentativa ou descarte)
```

As responsabilidades ficam divididas da seguinte forma:

- **Detecção (Opengrep).** É determinística e auditável. Cada achado está ligado a uma regra identificada e a uma CWE. O Opengrep é um *fork* do Semgrep distribuído sob a licença LGPL 2.1 (OPENGREP, 2026a). O projeto usa a versão 1.32.0 (CYLAB, 2026c; OPENGREP, 2026b).
- **Explicação e correção (LLM).** O modelo recebe o achado como contexto e não busca vulnerabilidades por conta própria. Sua função é produzir texto em português e uma reescrita do trecho.
- **Verificação (Opengrep).** O mesmo motor varre o *patch* proposto. A regra que gerou o achado deve deixar de disparar.

Essa separação restringe o LLM às tarefas de linguagem e reescrita e mantém a decisão sobre o que é vulnerável em um componente cujo comportamento pode ser inspecionado regra a regra. O Quadro 1 detalha entradas, saídas e o estado de implementação de cada componente.

Quadro 1 — Responsabilidades dos componentes da arquitetura (fonte: elaborado pelos autores; CYLAB, 2026a; 2026c)

Componente	Entrada	Saída	Decide o que é vulnerável?	Estado nesta entrega
Opengrep + regras Cylab (detecção)	código-fonte	achado	sim, por regra declarativa	implementado (regras `rules/` e `rules-gold/`)
LLM local via Ollama	achado + trecho	explicação pt-BR + *patch*	**não**	planejado; nesta entrega, explicação e *patch* vêm do conjunto de dados
Opengrep (re-varredura, *gate*)	*patch*	aprovado / reprovado	sim, pela mesma regra	implementado (`scripts/gate*.py`)
Conjunto de dados ouro	—	pares para treino e avaliação do LLM	—	FINAL v0.4-gold (576 pares)

O mesmo critério de aceite é usado em dois momentos. Na **construção do conjunto de dados**, `before` e `after` são escritos pela equipe e o *gate* filtra pares inconsistentes. Na **operação futura do sistema**, `before` é o código do usuário e `after` é o *patch* do LLM; o *gate* decide se o *patch* é entregue. Por isso, um conjunto em que 100% dos pares passam no *gate* é também o alvo objetivo de treino e avaliação do modelo.

2.3 Execução local

O sistema foi concebido para executar inteiramente na máquina do usuário. As razões são a privacidade do código, a ausência de custo por consulta, a reprodutibilidade e a possibilidade de uso em sala de aula ou sem conexão com a internet (CYLAB, 2026a). O

plano prevê servir o modelo por meio do Ollama, ferramenta de código aberto para execução local de LLMs (OLLAMA, 2026), com interface de linha de comando (CLI) e extensão para o VS Code. Esses componentes ainda **não** foram implementados.

2.4 Critério de aceite (*gate*) automático

Um par vulnerável–corrigido só é aceito se a regra Opengrep associada a ele produzir **mais de zero achados** no trecho vulnerável (`before`) e **zero achados** no trecho corrigido (`after`) (CYLAB, 2026c). Formalmente, para um par p com regra r:

$$\text{aceite}(p) \Longleftrightarrow \text{achados}_r(\text{before}_p) \geq 1 \;\wedge\; \text{achados}_r(\text{after}_p) = 0\,.$$

Para um conjunto D, a taxa de aprovação é a fração de pares aceitos, e a promoção a "ouro" exige taxa igual a 1:

$$\text{taxa}(D) = \frac{|\{p \in D : \text{aceite}(p)\}|}{|D|}, \qquad \text{ouro}(D) \Rightarrow \text{taxa}(D) = 1\,.$$

A primeira condição garante que a regra de fato reconhece a falha no código vulnerável (evita pares em que a regra nunca dispararia); a segunda, que a correção remove o padrão detectado. Um par com `before_hits = 0` é rejeitado mesmo que o `after` esteja correto, porque não serviria para ensinar o modelo a responder a um achado.

O critério é implementado em `scripts/gate.py` e, nas versões usadas para o conjunto ouro, em `scripts/gate_draft.py` (corte v0.3) e `scripts/gate_gold.py` (corte v0.4). Ambos resolvem cada regra procurando primeiro no diretório de regras endurecidas (`rules-gold/<regra>.yaml`) e, na ausência dela, em `rules/<regra>.yaml` (CYLAB, 2026g; CYLAB, 2026n). O relatório por par registra `before_hits`, `after_hits` e `passed`. Há ainda um validador de esquema (`scripts/validate_dataset.py`) que verifica campos obrigatórios, a lista de linguagens permitidas, o formato da CWE e do campo `source`, a licença, a existência do arquivo de regra, a diferença entre `before` e `after` e a validade do *split* (CYLAB, 2026c).

O critério converte a pergunta "o *patch* está correto?" em um teste objetivo e reprodutível, mas sua cobertura é parcial. Ele garante que a falha **detectada pela regra** desaparece; não garante a correção funcional do código (ver Seção 5).

2.5 Regras endurecidas (`-gold`)

As regras originais (`rules/`) eram propositalmente simples e calibradas para a semente. A documentação do *gate* já recomendava endurecê-las antes de escalar o conjunto (CYLAB, 2026c), pois uma regra frouxa aceita *patches* que apenas trocam a forma textual do código. Foram então criadas regras endurecidas em `rules-gold/`:

- **C/C++:** padrões estruturais com metavariáveis, `pattern-not`, `pattern-inside` e `metavariable-regex`, sem modo *taint* (ver Seção 5) (CYLAB, 2026d). Exemplo:

`rules-gold/cylab.c.strcpy-strcat-gold.yaml` casa `strcpy($D, $S)` e `strcat($D, $S)` como estrutura sintática, e não como busca textual.

- **Bash:** expressões regulares restritas a linhas não comentadas, porque o suporte estrutural do Opengrep a Bash é limitado (CYLAB, 2026d).
- **Python/JavaScript/TypeScript e Java/PHP/C#:** regras em modo *taint*, que exigem que um valor vindo de uma fonte externa alcance um *sink* (por exemplo, `execute` em consultas SQL). Em Java/PHP/C#, 32 regras estão em modo *taint* ativo; regras de criptografia, segredos, cabeçalhos, XXE e libxml permaneceram em regex por decisão de projeto (CYLAB, 2026a).
- **SQL:** 14 regras com regex mais restrita, pois o Opengrep não oferece *taint* para SQL (CYLAB, 2026a).

A regra de resolução é única para todo o conjunto: se existe versão endurecida, ela prevalece; caso contrário, usa-se a regra original. O repositório contém hoje 134 arquivos em `rules-gold/` e 228 em `rules/`.

2.6 Detecção de quase-duplicatas

Para evitar que o conjunto contenha variações triviais de um mesmo modelo de geração (*template*), aplicou-se detecção de quase-duplicatas baseada na similaridade de Jaccard entre conjuntos de *shingles* (n-gramas de *tokens*), técnica clássica de estimativa de semelhança entre documentos (BRODER, 1997). Sendo $S_n(x)$ o conjunto de n-gramas de *tokens* do trecho normalizado x, a similaridade entre dois pares a e b é:

$$J(a,b) = \frac{|S_n(\text{before}_a) \cap S_n(\text{before}_b)|}{|S_n(\text{before}_a) \cup S_n(\text{before}_b)|}.$$

A normalização remove comentários e substitui literais de texto, números e identificadores que não são palavras-chave por marcadores fixos, de modo que dois trechos que diferem apenas em nomes de variáveis ou constantes sejam reconhecidos como o mesmo *template*. A implementação do projeto (`scripts/gold_near_dup.py`) agrupa os pares dentro de um mesmo estrato (linguagem + CWE) por *union-find* com ligação simples. Dois pares são ligados quando (a) os trechos `before` normalizados são idênticos ou (b) $J \geq \tau$ entre 4-gramas de *tokens*, com limiar padrão $\tau = 0{,}85$. De cada grupo mantêm-se até dois representantes, o medoide e o membro mais distante (CYLAB, 2026d; CYLAB, 2026f). A deduplicação é sempre feita **antes** da partição treino/*holdout*, para que quase-clones não fiquem em lados opostos da partição.

2.7 Onda de realismo (*buffer* v0.4)

A principal limitação do corte v0.3 é o tamanho dos trechos: a mediana do `before` é de 6 linhas (ver Seção 4). Para aproximar o conjunto de código de produção sem perder as garantias do *gate*, executou-se uma onda adicional de curadoria, chamada *buffer* de

realismo, com as seguintes regras de método (CYLAB, 2026o):

1. **Escopo por frente.** Cada frente de trabalho escreveu do zero pares maiores nas suas linguagens: Java/PHP/C# (18), SQL (31), C/C++/Bash (36) e Python/JavaScript/TypeScript (15), totalizando 100 pares. Os trechos `before` têm entre 26 e 52 linhas, com mediana de 35 (contagem direta em CYLAB, 2026m).
2. **Imutabilidade do corte anterior.** O arquivo da v0.3-gold foi lido somente para comparação; seu SHA-256 é o mesmo antes e depois da fusão (`f37e1316…25ea3` → idêntico) (CYLAB, 2026o).
3. **Identificadores.** Nenhuma colisão de `id` com a v0.3 (0 colisões; 0 renomeações).
4. **Quase-duplicatas.** Comparou-se cada par novo com os demais novos e com os 476 pares da v0.3 no mesmo estrato (linguagem + CWE), por `SequenceMatcher` ou Jaccard de 5-gramas ≥ 0,85, e também pelo método global (`gold_near_dup.py`, Jaccard de 4-gramas normalizados ≥ 0,85). Houve um único grupo interno de 2 pares SQL, ambos mantidos por terem formas de `before` e `after` distintas, e nenhum par novo se agrupou com a v0.3; a maior similaridade observada contra a v0.3 foi 0,376 em 792 comparações (CYLAB, 2026o).
5. ***Gate* e validação.** O *buffer* passou em 100/100 no *gate* com `rules-gold` e no validador de esquema (OK: 100 pares, 12 *holdout*) (CYLAB, 2026o).
6. **Partição.** Os pares de Python/JS/TS mantiveram a partição proposta pela frente; os demais foram para o *holdout* quando `int(sha256(id)) mod 10 = 0`, o que resultou em 88 pares de treino e 12 de *holdout*.

O corte FINAL v0.4-gold é a união exata da v0.3-gold (476) com o *buffer* (100), re-verificada como um todo: 576/576 no *gate* e validador OK (CYLAB, 2026n).

2.8 Técnicas de adaptação previstas

O ajuste fino planejado usa adaptação de baixo posto (LoRA), que congela os pesos do modelo pré-treinado e treina apenas matrizes de decomposição de baixo posto inseridas nas camadas do Transformer (HU et al., 2021). Prevê-se também a variante QLoRA, que retropropaga gradientes através de um modelo base quantizado em 4 bits (DETTMERS et al., 2023). O modelo base planejado para a primeira rodada é o Qwen2.5-Coder-14B, da família Qwen2.5-Coder (HUI et al., 2024; QWEN TEAM, 2024). Esses elementos são **plano de trabalho**; nenhum treinamento foi executado.

3 MATERIAIS: O CONJUNTO DE DADOS

3.1 Esquema do par

Cada registro do arquivo JSONL descreve um par, com chaves em ordem fixa (CYLAB, 2026b). O Quadro 2 resume o esquema.

Quadro 2 — Esquema de um par do conjunto Cylab (fonte: `SCHEMA.md`)

Campo	Conteúdo
`id`	identificador único do par
`lang`	`python`, `javascript`, `typescript`, `java`, `php`, `csharp`, `c`, `cpp`, `sql` ou `bash`
`cwe`	`CWE-<n>`
`source`	`sintetico-cylab` `https://cwe.mitre.org/data/definitions/<n>.html`
`license`	`CC-BY-4.0`
`opengrep_rule`	`cylab.<lang>.<slug>`, não nulo, com arquivo de regra correspondente
`before`	trecho vulnerável completo
`after`	trecho corrigido completo (não é um *diff*)
`explanation_pt`	explicação em português do Brasil que cita a CWE
`split`	`train` ou `holdout`

Exemplo (par `cylab-bill-python-89-0002`, CWE-89, regra `cylab.python.sqli-fstring-execute`, *split* `holdout`): o `before` monta a consulta SQL com `.format()` sobre um valor recebido do chamador, e o `after` usa parâmetro vinculado (`?`). A `explanation_pt` registrada é: "A query monta SQL com interpolação de string usando o valor de status vindo do chamador, o que permite injeção de SQL (CWE-89). A correção passa o valor como parâmetro vinculado, e o driver o trata sempre como dado." (CYLAB, 2026e). A falha corresponde à CWE-89 (MITRE CORPORATION, 2026b).

3.2 Proveniência e licença

Todos os pares foram escritos do zero para a Cylab, sem leitura, importação ou conversão de dados de terceiros ou de lotes anteriores (CYLAB, 2026h). Não há dados do projeto VigIA (CYLAB, 2026a); uma busca pelo termo nos 576 registros do arquivo FINAL v0.4-gold não retornou ocorrências (CYLAB, 2026m). Os 576 registros declaram a licença CC-BY-4.0 (CREATIVE COMMONS, 2026; CYLAB, 2026m).

3.3 Processo de curadoria (funil)

O conjunto ouro resultou de uma sequência de versões:

1. **v0.1 (semente).** Foram 38 pares e 34 regras, usados para exercitar o *pipeline*; o *gate* registrou 38/38 (CYLAB, 2026c; CYLAB, 2026h).
2. **v0.2 comercial.** Foram 2.191 pares em 10 linguagens, voltados à meta de cerca de 2 mil pares (CYLAB, 2026a; arquivo `data/cylab-commercial-v0.2.jsonl`).
3. **Diagnóstico de quase-duplicatas.** A análise global da v0.2 (limiar 0,85, estrato linguagem + CWE) encontrou 248 grupos, que reuniam 2.138 dos 2.191 pares. Foram marcados 1.751 pares para descarte e 440 mantidos (CYLAB, 2026f). Na fatia C/C++/Bash, entraram 602 pares e restaram 182 (420 descartados, 65 grupos). Os cortes em 0,80, 0,85, 0,90, 0,95 e 0,99 produziram os mesmos 182 pares, o que indica que esses grupos eram clones exatos de *template* (CYLAB, 2026d).
4. **Núcleos por frente de trabalho.** Python/JS/TS: 151 pares (`contributions/sam-py-js/gold_py_js_ts.jsonl`); nessa frente, a deduplicação usou `difflib.SequenceMatcher` (razão ≥ 0,85 no `before` normalizado, estrato linguagem + CWE + regra), e não o Jaccard de *shingles* do script global (`contributions/sam-py-js/gold_near_dup_report.json`) (CYLAB, 2026k). Java/PHP/C#/SQL: 92 pares, com 92/92 no *gate* com regras de *taint* (`contributions/markinho-java-php-sql-shell/gold_taint_progress.json`). C/C++/Bash: 182 pares, com 182/182 no *gate* com regras `-gold` (CYLAB, 2026d). A união soma 425 pares, sem identificadores repetidos entre núcleos (CYLAB, 2026g).
5. **Complemento.** Do conjunto candidato global entraram 51 pares que não eram quase-duplicatas dos já mantidos; 181 candidatos foram descartados por serem quase-duplicatas (critério: Jaccard de 5-gramas ≥ 0,9 no estrato linguagem + CWE + regra) (CYLAB, 2026g).
6. **Corte FINAL v0.3-gold (corte anterior).** São 476 pares, cortados em 27/10/2026 às 11h04 (UTC−3), com SHA-256 original `fe2d3846…09ca35` registrado em `data/gold-progress.json` (CYLAB, 2026i). Após a normalização do campo `source` em 28/10/2026, o SHA-256 passou a ser `f37e1316…25ea3`; nenhum outro campo foi alterado.
7. **Corte FINAL v0.4-gold (atual).** São 576 pares: os 476 da v0.3-gold, sem alterações, mais 100 do *buffer* de realismo (Seção 2.7). O corte foi feito em 28/10/2026 às 09h08 (UTC−3), com SHA-256 `ea328aef…9806c`, registrado em `data/v0.4-gold-cut-meta.json` (CYLAB, 2026n).

Os limiares e os métodos diferem entre as frentes (Jaccard 0,85 no relatório global e na fatia C/C++/Bash; `SequenceMatcher` ≥ 0,85 no núcleo Python/JS/TS; Jaccard 0,82 no núcleo Java/PHP/C#/SQL; Jaccard 0,9 no complemento), conforme registrado nos

respectivos artefatos. O arquivo `data/gold-progress.json` também guarda uma rodada intermediária com 722 candidatos (limiar 0,9) e 706/722 aprovações no *gate*. Essa rodada foi superada pelo corte final e não integra os resultados.

3.4 Partição treino/*holdout*

A política de *holdout* registrada no manifesto é a seguinte: quando o identificador consta da proposta global de partição (15% por estrato linguagem + regra, ordenado por hash SHA-256 com sal fixo), adota-se essa atribuição; nos demais casos, usa-se `sha256(sal|lang|cwe|id) mod 100 < 10` (CYLAB, 2026g; CYLAB, 2026d). Pela proposta, 359 pares tiveram o *split* definido e 117 pelo hash de 10% (CYLAB, 2026g). Na v0.4, os 75 pares de *holdout* da v0.3 foram preservados e o *buffer* acrescentou 12, totalizando 87 (Seção 2.7). A deduplicação foi feita antes da partição, o que reduz o risco de vazamento de quase-clones entre treino e *holdout*.

4 RESULTADOS

Os números do corte atual vêm de `data/cylab-commercial-v0.4-gold.jsonl`, `data/v0.4-gold-cut-meta.json`, `data/v0.4-realism-merge-meta.json` e `data/gate-report-v0.4-gold.json` (CYLAB, 2026m; 2026n; 2026o; 2026p). Os do corte anterior vêm de `data/cylab-commercial-v0.3-gold.jsonl`, `data/gold-progress.json` (chave `gold_final`), `data/v0.3-gold-draft-manifest.json` e `data/gate-report-v0.3-gold-draft-retake.json` (CYLAB, 2026e; 2026g; 2026i; 2026j). **Não se trata de métricas de modelo**: os números medem a consistência do conjunto de dados em relação às regras.

4.1 Visão geral do corte atual (FINAL v0.4-gold)

- **Tamanho:** 576 pares; status oficial **FINAL v0.4-gold** = v0.3-gold (476) + *buffer* de realismo (100) (CYLAB, 2026n).
- ***Gate*:** 576/576 aprovações (`scripts/gate_gold.py --rules-dir rules-gold`), com relatório por par (CYLAB, 2026p). Dos 576 pares, 468 foram verificados por regras resolvidas em `rules-gold/` e 108 pelas regras originais em `rules/`.
- **Validador:** OK, 576 pares, 87 de *holdout*; 0 colisões de identificador (CYLAB, 2026n).
- **Partição:** 489 pares de treino e 87 de *holdout*.
- **Diversidade:** 10 linguagens, 43 CWEs distintas e 178 identificadores de regra distintos. As CWEs mais frequentes são CWE-89 (79 pares), CWE-78 (68), CWE-120 (57), CWE-798 (47), CWE-79 (36) e CWE-22 (35) (contagem direta em CYLAB, 2026m; MITRE CORPORATION, 2026b–g).

- **Tamanho dos trechos:** a mediana do `before` permanece em 6 linhas no conjunto completo, mas o máximo passou de 16 para 52 linhas; no *buffer* de realismo, a mediana é de 35 linhas (CYLAB, 2026m).

Tabela 1 — Pares por linguagem no FINAL v0.4-gold (fonte: `data/v0.4-gold-cut-meta.json`; `data/v0.4-realism-merge-meta.json`)

Linguagem	v0.3-gold	+ Realismo	v0.4-gold	Treino	*Holdout*
Python	68	6	74	66	8
JavaScript	63	5	68	60	8
TypeScript	33	4	37	30	7
Java	34	6	40	29	11
PHP	33	6	39	34	5
C#	27	6	33	30	3
C	55	12	67	54	13
C++	74	12	86	76	10
SQL	34	31	65	57	8
Bash	55	12	67	53	14
Total	**476**	**100**	**576**	**489**	**87**

4.2 Corte anterior (FINAL v0.3-gold)

- **Tamanho:** 476 pares; status **FINAL v0.3-gold**, corte anterior e base integral da v0.4 (CYLAB, 2026i).
- ***Gate*:** 476/476 aprovações na re-verificação de 27/10/2026. O relatório por par traz `before_hits`, `after_hits` e `passed` para cada um dos 476 registros, todos com `passed = true` (CYLAB, 2026j).
- **Partição:** 401 pares de treino e 75 de *holdout* (CYLAB, 2026i).
- **Linguagens:** as 10 linguagens previstas estão presentes, sem nenhuma ausente (`missing_langs: []`) (CYLAB, 2026g).
- **Diversidade:** 43 CWEs distintas e 178 identificadores de regra distintos no arquivo final (contagem direta em CYLAB, 2026e). As CWEs mais frequentes são CWE-89 (56 pares), CWE-78 (56), CWE-120 (52), CWE-798 (40), CWE-79 (30) e CWE-22 (27) (MITRE CORPORATION, 2026b–g).
- **Tamanho dos trechos:** a mediana do `before` é de 6 linhas e o máximo é de 16 linhas (CYLAB, 2026e).

4.3 Distribuição por linguagem e partição no corte v0.3

Tabela 2 — Pares por linguagem e partição no FINAL v0.3-gold (fonte: `data/cylab-commercial-v0.3-gold.jsonl`; `data/v0.3-gold-draft-manifest.json`)

Linguagem	Treino	*Holdout*	Total
Python	61	7	68
JavaScript	56	7	63
TypeScript	27	6	33
Java	26	8	34
PHP	28	5	33
C#	24	3	27
C	45	10	55
C++	64	10	74
SQL	29	5	34
Bash	41	14	55
Total	**401**	**75**	**476**

4.4 Origem dos pares

Tabela 3 — Composição do FINAL v0.4-gold por origem (fonte: `data/v0.3-gold-draft-manifest.json`, chave `by_source`; `data/v0.4-realism-merge-meta.json`, chave `by_owner`)

Origem	Linguagens	Pares
Núcleo Python/JS/TS (`contributions/sam-py-js/`)	Python, JavaScript, TypeScript	151
Núcleo Java/PHP/C#/SQL (`contributions/markinho-java-php-sql-shell/`)	Java, PHP, C#, SQL	92
Núcleo C/C++/Bash (`contributions/elon-c-cpp-sql-bash/`)	C, C++, Bash	182
Complemento do candidato global v0.3 (não quase-duplicatas)	várias	51
Subtotal v0.3-gold		*476*
Buffer de realismo — frente Java/PHP/C#	Java, PHP, C#	18
Buffer de realismo — frente SQL	SQL	31
Buffer de realismo — frente C/C++/Bash	C, C++, Bash	36
Buffer de realismo — frente Python/JS/TS	Python, JavaScript, TypeScript	15
Total v0.4-gold		**576**

4.5 Regras utilizadas na verificação

Na re-verificação do corte v0.3, 314 dos 476 pares foram verificados por regras resolvidas em `rules-gold/` (endurecidas) e 162 pelas regras originais em `rules/` (CYLAB, 2026g; CYLAB, 2026j). No corte v0.4, verificado com `--rules-dir rules-gold`, a proporção passou a 468 de 576 em `rules-gold/` e 108 em `rules/`; os 100 pares do *buffer* usam todos regras endurecidas (CYLAB, 2026p).

4.6 Exemplo verificado

O par `cylab-bill-c-120-0613` (C, CWE-120, *holdout*, regra `cylab.c.strcpy-strcat-gold`) copia uma entrada externa com `strcpy` para um *buffer* de 32 bytes. O `after` substitui a cópia por `snprintf` limitado ao tamanho do destino. Segundo o roteiro de demonstração, a regra produz 2 achados no `before` e 0 no `after` (CYLAB, 2026a; script `seminar/demo/demo_pair.sh`). A falha corresponde à CWE-120 (MITRE CORPORATION, 2026d).

5 DISCUSSÃO E LIMITAÇÕES

Volume e diversidade. O conjunto final (576) é menor que a v0.2 (2.191) por decisão de projeto. Na fatia C/C++/Bash, 420 dos 602 pares eram clones de *template* (CYLAB, 2026d). Treinar com dezenas de variações de uma mesma chamada tenderia a ensinar o modelo a memorizar o *template* em vez de reconhecer a falha. Optou-se por pares distintos e verificados.

Análise de fluxo (*taint*) em C/C++. Nos testes do projeto, o modo *taint* do Opengrep 1.32 não produziu achados em C/C++. Por isso, as regras `-gold` dessas linguagens são estruturais e não acompanham o fluxo de dados (CYLAB, 2026d). Duas regras de C++ continuam em regex ancorada em declaração (`cpp.hardcoded-key-literal` e `cpp.dangling-cstr`), porque o *parser* não casou determinadas declarações (CYLAB, 2026d). Em Bash, as regras usam regex restrita. Em SQL não há *taint* (CYLAB, 2026a).

Pares sintéticos e curtos. Os trechos são, em sua maioria, funções curtas (mediana de 6 linhas no `before`) e não representam código de produção com múltiplos arquivos e dependências (CYLAB, 2026m). O *buffer* de realismo da v0.4 atenua o problema, com 100 pares de 26 a 52 linhas, mas eles são 17% do conjunto, continuam sintéticos e de arquivo único. A generalização para código real segue como a principal questão em aberto.

O *gate* não equivale a correção semântica. Zero achados no `after` mostram que a regra deixou de disparar, e não que o *patch* preserve o comportamento esperado em qualquer contexto. Há também o risco de um modelo treinado aprender a satisfazer a regra em vez de corrigir a falha (*overfitting* à regra). As medidas previstas são a deduplicação (já aplicada), o *holdout* estratificado, a revisão humana e, futuramente, testes funcionais (CYLAB, 2026a).

Metadados `source`. Em 28/10/2026 os 65 pares cujo `source` não trazia a URL da CWE foram normalizados para o formato do esquema (`sintetico-cylab https://cwe.mitre.org/data/definitions/<n>.html`); o validador passa com **0 erros** no FINAL v0.3 e no FINAL v0.4 (CYLAB, 2026e; CYLAB, 2026n; `data/source-fix-2026-10-28.md`).

Ausência de modelo. Não existe modelo treinado nem ambiente Ollama configurado no ambiente do projeto (CYLAB, 2026a). Nenhuma afirmação deste relatório se refere a desempenho de modelo.

6 TRABALHOS FUTUROS

1. ~~**Correção dos metadados `source`.**~~ Feito em 28/10/2026 (`data/source-fix-2026-10-28.md`); validador OK.

2. **Ajuste fino.** Aplicar LoRA/QLoRA (HU et al., 2021; DETTMERS et al., 2023) ao Qwen2.5-Coder-14B (HUI et al., 2024) e, depois, a um modelo maior da família Qwen, indicado no roteiro como "Qwen3.8-27B". A denominação e a versão exatas desse checkpoint ainda precisam ser confirmadas (CYLAB, 2026a).
3. **Avaliação no *holdout*.** O critério principal será objetivo: a proporção de *patches* gerados pelo modelo que passam no *gate* nos 87 pares de *holdout* da v0.4 (75 herdados da v0.3 e 12 do *buffer* de realismo), reportada separadamente para os dois grupos, complementada por revisão humana das explicações (CYLAB, 2026a).
4. **Empacotamento local.** Servir o modelo via Ollama (OLLAMA, 2026), com CLI e extensão para o VS Code.
5. **Realismo dos dados.** ~~Incluir funções maiores~~ primeira onda feita na v0.4 (100 pares, 26–52 linhas). Falta incluir casos com múltiplos arquivos e reavaliar o *taint* em C/C++ quando houver suporte no Opengrep.
6. **Requisitos de *hardware*.** Medir após o ajuste fino; nenhuma estimativa é apresentada nesta entrega.

7 CONCLUSÃO

Esta primeira entrega definiu uma arquitetura em que a detecção de vulnerabilidades permanece com um motor determinístico e auditável (Opengrep) e o LLM local fica restrito a explicar em português e propor a correção, que é novamente verificada pelo mesmo motor. Como base para treinar e avaliar esse componente, construiu-se o conjunto de dados ouro FINAL v0.4-gold: 576 pares originais sob licença CC-BY-4.0, em 10 linguagens, com 576/576 aprovações no critério de aceite automático e partição de 489 pares de treino e 87 de *holdout*. O conjunto parte do corte anterior FINAL v0.3-gold (476 pares, obtidos a partir de 2.191 pares por meio da remoção de quase-duplicatas) e acrescenta um *buffer* de 100 pares maiores, sem quase-duplicatas em relação à v0.3. Os artefatos estão publicados em https://github.com/ArthurAlcausa/cylab-dataset. As limitações (pares sintéticos predominantemente curtos, *gate* que não garante correção semântica e ausência de *taint* em C/C++) estão documentadas; os metadados `source` já foram normalizados. A próxima etapa deve responder se um modelo local ajustado com esses pares gera *patches* que passam no *gate* no conjunto de *holdout*.

REFERÊNCIAS

Literatura e ferramentas

BRODER, A. Z. On the resemblance and containment of documents. *In*: COMPRESSION AND COMPLEXITY OF SEQUENCES, 1997, Salerno. **Proceedings** [...]. Los Alamitos: IEEE Computer Society, 1997. p. 21–29. DOI: 10.1109/SEQUEN.1997.666900.

CREATIVE COMMONS. **Attribution 4.0 International (CC BY 4.0)**. [*S. l.*]: Creative Commons, [2026]. Disponível em: https://creativecommons.org/licenses/by/4.0/. Acesso em: 27 out. 2026.

DETTMERS, T.; PAGNONI, A.; HOLTZMAN, A.; ZETTLEMOYER, L. **QLoRA**: efficient finetuning of quantized LLMs. arXiv:2305.14314, 2023. Disponível em: https://arxiv.org/abs/2305.14314. Acesso em: 27 out. 2026.

HU, E. J.; SHEN, Y.; WALLIS, P.; ALLEN-ZHU, Z.; LI, Y.; WANG, S.; WANG, L.; CHEN, W. **LoRA**: low-rank adaptation of large language models. arXiv:2106.09685, 2021. Disponível em: https://arxiv.org/abs/2106.09685. Acesso em: 27 out. 2026.

HUI, B. *et al*. **Qwen2.5-Coder technical report**. arXiv:2409.12186, 2024. Disponível em: https://arxiv.org/abs/2409.12186. Acesso em: 27 out. 2026.

MITRE CORPORATION. **CWE – Common Weakness Enumeration**. Bedford: MITRE, [2026a]. Disponível em: https://cwe.mitre.org/. Acesso em: 27 out. 2026.

MITRE CORPORATION. **CWE-89**: improper neutralization of special elements used in an SQL command ('SQL injection'). Bedford: MITRE, [2026b]. Disponível em: https://cwe.mitre.org/data/definitions/89.html. Acesso em: 27 out. 2026.

MITRE CORPORATION. **CWE-78**: improper neutralization of special elements used in an OS command ('OS command injection'). Bedford: MITRE, [2026c]. Disponível em: https://cwe.mitre.org/data/definitions/78.html. Acesso em: 27 out. 2026.

MITRE CORPORATION. **CWE-120**: buffer copy without checking size of input ('classic buffer overflow'). Bedford: MITRE, [2026d]. Disponível em: https://cwe.mitre.org/data/definitions/120.html. Acesso em: 27 out. 2026.

MITRE CORPORATION. **CWE-798**: use of hard-coded credentials. Bedford: MITRE, [2026e]. Disponível em: https://cwe.mitre.org/data/definitions/798.html. Acesso em: 27 out. 2026.

MITRE CORPORATION. **CWE-79**: improper neutralization of input during web page generation ('cross-site scripting'). Bedford: MITRE, [2026f]. Disponível em: https://cwe.mitre.org/data/definitions/79.html. Acesso em: 27 out. 2026.

MITRE CORPORATION. **CWE-22**: improper limitation of a pathname to a restricted directory ('path traversal'). Bedford: MITRE, [2026g]. Disponível em: https://cwe.mitre.org/data/definitions/22.html. Acesso em: 27 out. 2026.

OLLAMA. **Ollama**: get up and running with large language models locally. [*S. l.*], [2026]. Disponível em: https://ollama.com e https://github.com/ollama/ollama. Acesso em: 27 out. 2026.

OPENGREP. **Opengrep**: static code analysis engine to find security issues in code. [*S. l.*], [2026a]. Repositório GitHub (*fork* do Semgrep, licença LGPL 2.1). Disponível em: https://github.com/opengrep/opengrep. Acesso em: 27 out. 2026.

OPENGREP. **Release v1.32.0**. [*S. l.*], [2026b]. Disponível em: https://github.com/opengrep/opengrep/releases/tag/v1.32.0. Acesso em: 27 out. 2026.

QWEN TEAM. **Qwen2.5-Coder**. [*S. l.*]: Alibaba Group, 2024. Repositório GitHub. Disponível em: https://github.com/QwenLM/Qwen2.5-Coder. Acesso em: 27 out. 2026.

SEMGREP. **Semgrep**: lightweight static analysis for many languages. [*S. l.*], [2026]. Repositório GitHub. Disponível em: https://github.com/semgrep/semgrep. Acesso em: 27 out. 2026.

Artefatos do Projeto Cylab (documentos técnicos)

Todos os caminhos são relativos à raiz do repositório público do conjunto de dados (CYLAB, 2026l).

CYLAB. **Roteiro do seminário (PUC-Campinas, banca)**. Campinas, 28 out. 2026a. Documento técnico. Caminho: `seminar/ROTEIRO_SEMINARIO_PUC.md`.

CYLAB. **Schema (JSONL)**. Campinas, 26 out. 2026b. Documento técnico. Caminho: `SCHEMA.md`.

CYLAB. **Gates**. Campinas, 26 out. 2026c. Documento técnico. Caminho: `GATES.md` (scripts `scripts/gate.py`, `scripts/gate_draft.py`, `scripts/validate_dataset.py`).

CYLAB. **Elon — ouro C/C++/Bash**: relatório. Campinas, 27 out. 2026d. Documento técnico. Caminho: `contributions/elon-c-cpp-sql-bash/REPORT.md`.

CYLAB. **Conjunto de dados Cylab commercial v0.3-gold (FINAL)**. Campinas, 27 out. 2026e. Arquivo JSONL, 476 registros, licença CC-BY-4.0. Caminho: `data/cylab-commercial-v0.3-gold.jsonl`.

CYLAB. **Relatório de quase-duplicatas da v0.2**. Campinas, 27 out. 2026f. Arquivo JSON. Caminho: `data/gold-near-dup-report.json` (gerado por `scripts/gold_near_dup.py`).

CYLAB. **Relatório de quase-duplicatas do núcleo Python/JavaScript/TypeScript**. Campinas, 27 out. 2026k. Arquivo JSON. Caminho: `contributions/sam-py-js/gold_near_dup_report.json` (método `difflib.SequenceMatcher`, limiar 0,85).

CYLAB. **Manifesto do v0.3-gold**. Campinas, 27 out. 2026g. Arquivo JSON. Caminho: `data/v0.3-gold-draft-manifest.json`.

CYLAB. **Cylab Dataset v0.1**: README e CHANGELOG. Campinas, 26 out. 2026h. Documento técnico. Caminhos: `README.md`, `CHANGELOG.md`.

CYLAB. **Registro de progresso do ouro**. Campinas, 27 out. 2026i. Arquivo JSON, chave `gold_final`. Caminho: `data/gold-progress.json`.

CYLAB. **Relatório de re-verificação do gate v0.3-gold**. Campinas, 27 out. 2026j. Arquivo JSON, 476 registros. Caminho: `data/gate-report-v0.3-gold-draft-retake.json`.

CYLAB. **cylab-dataset**: conjunto de dados ouro de pares vulnerável–corrigido, regras Opengrep e *scripts* de verificação. Campinas: Projeto Cylab, PUC-Campinas, 2026l. Repositório GitHub (dados sob CC-BY-4.0). Disponível em: https://github.com/ArthurAlcausa/cylab-dataset. Acesso em: 28 out. 2026.

CYLAB. **Conjunto de dados Cylab commercial v0.4-gold (FINAL)**. Campinas, 28 out. 2026m. Arquivo JSONL, 576 registros, licença CC-BY-4.0. Caminho: `data/cylab-commercial-v0.4-gold.jsonl`.

CYLAB. **Metadados do corte v0.4-gold**. Campinas, 28 out. 2026n. Arquivo JSON. Caminho: `data/v0.4-gold-cut-meta.json` (*script* `scripts/gate_gold.py`).

CYLAB. **Metadados da fusão do *buffer* de realismo v0.4**. Campinas, 28 out. 2026o. Arquivos JSON. Caminhos: `data/v0.4-realism-merge-meta.json` e `data/near-dup-v0.4-realism-draft.json` (*script* `scripts/build_v04_realism_draft.py`).

CYLAB. **Relatório do gate v0.4-gold**. Campinas, 28 out. 2026p. Arquivo JSON, 576 registros. Caminho: `data/gate-report-v0.4-gold.json`.

Artefatos complementares citados no texto: `contributions/sam-py-js/gold_py_js_ts.jsonl`; `contributions/markinho-java-php-sql-shell/gold_java_php_csharp_sql.jsonl` e `gold_taint_progress.json`; `contributions/elon-c-cpp-sql-bash/elon_gold.jsonl` e `holdout_split_proposal.py`; `rules-gold/cylab.c.strcpy-strcat-gold.yaml`; `seminar/demo/demo_pair.sh`; `data/cylab-commercial-v0.2.jsonl`; `data/cylab-commercial-v0.4-realism-DRAFT.jsonl`; `seminar/assets/cylab-arquitetura.svg`, `.png` e `.mmd` (Figura 1).